Crossword Puzzle Generator Using Backtracking: Design, Repair, and Analysis

1st Dipika Satapathy
Department of CSE
K L E F,
Vaddeswaram, Guntur
dipika.satapathy@gmail.com

***Abstract*—The Crossword Puzzle Generator Using Backtracking is a Python-based project that automatically creates crossword puzzles by placing words into a grid according to specified constraints. The project uses backtracking, a systematic problem-solving technique that tries candidate solutions and retreats whenever a choice cannot lead to a valid result. It also focuses on repairing an existing generator by identifying and correcting errors in word placement, grid boundaries, overlapping letters, and constraint checking. The repaired system aims to generate valid puzzles efficiently and reduce placement errors. The project demonstrates the practical application of backtracking, recursion, constraint satisfaction, and algorithm design.**

***Index Terms*—backtracking, constraint satisfaction, crossword generation, Python, recursion.**

I. Introduction

A crossword puzzle consists of a grid in which words are placed horizontally or vertically [1] [2]. The words must satisfy certain conditions, such as matching letters at intersections and remaining within the grid boundaries. Creating a puzzle manually is time-consuming; this project therefore develops an automated crossword puzzle generator in Python. The backtracking algorithm attempts to place each word in the grid. If a word cannot be placed correctly, the algorithm removes the previous placement and tries another possibility. This continues until a valid puzzle is generated or all possible placements have been explored. The term "repaired" indicates that an existing implementation is corrected so that it handles invalid placements and other programming errors properly.

- Problem Statement
  The goal is to develop and repair a Python-based crossword puzzle generator using backtracking. The system must place a given set of words into a grid while satisfying constraints on grid boundaries, matching letters, and valid word placement. The program should undo invalid placements and search for alternatives until it generates a valid puzzle or determines that no solution exists under the given constraints.
- Objectives
  - To generate crossword puzzles automatically using Python.
  - To implement the backtracking algorithm for word placement.
  - To ensure that words remain within the grid boundaries.
  - To allow intersecting words only when their letters match.
  - To repair errors in the existing puzzle generator.
  - To improve the correctness and reliability of puzzle generation.
  - To understand recursion and constraint-satisfaction techniques.
- Scope
  The project is suitable for educational applications and algorithm demonstrations. Its scope includes creating a crossword grid, accepting a predefined list of words, searching for valid positions for each word, checking horizontal and vertical placements, undoing invalid choices through backtracking, and displaying the generated puzzle. The current version may use a predefined word list and grid size; graphical interfaces, automatic clue generation, and difficulty levels are left for future versions.

II. TECHNOLOGIES AND REQUIREMENTS

The technologies used are summarized in Table I.

Component	Technology
Language	Python
Algorithm	Backtracking
Concepts	Recursion, constraint satisfaction
Data structures	Lists, strings, 2D grid
Data structures	Lists, strings, 2D grid
Environment	VS Code, PyCharm, or IDLE
Testing	Sample inputs and expected-output checks

Hardware: a computer or laptop with at least 4 GB of RAM (recommended) and a standard keyboard and display. Software: Windows or Linux, Python 3.x, and a Python editor or IDE. No external Python library is required for a basic implementation.

III. SYSTEM DESIGN

The system follows a simple sequence: (1) initialize the crossword grid; (2) read the list of words; (3) select a word that has not yet been placed; (4) identify possible horizontal or vertical positions; (5) check whether a position satisfies the constraints; (6) place the word if the position is valid;

(7) recursively attempt to place the remaining words; (8) if a later placement fails, remove the current word and try another position; and (9) display the completed grid if a solution is found.

The main modules are:

- Grid Initialization: creates an empty two-dimensional grid.
- Word Placement: attempts to position each word horizontally or vertically.
- Constraint Checking: checks grid boundaries and compatibility of intersecting letters.
- Backtracking: removes a previously placed word when the current choice prevents a solution.
- Puzzle Display: prints the final grid or reports that no solution was found

IV. ALGORITHM

The input is the grid dimensions and a list of words; the output is a valid crossword grid, if a solution exists. The procedure is:

- Create an empty grid
- Sort the words by length, if the implementation uses this optimization.
- Select the next word.
- Generate possible positions and orientations.
- Check whether each position is valid.
- If valid, place the word.
- Recursively place the next word.
- If the recursive attempt fails, remove the word and try another possible position.
- If all words are placed, return success.
- If all choices fail, return failure.
- Display the result.

V. WORKING PRINCIPLE

Backtracking follows the principle of try, check, and undo. For example, suppose the word list contains CAT, CAR, and ART. The program first places CAT and then searches for a valid position for CAR. If CAR intersects CAT, the common letter must match. The program then tries to place ART; if ART cannot be placed without violating the rules, the program removes the previous placement and tries another position. The process continues until the words fit the constraints or no valid arrangement is found.

VI. REPAIR AND IMPROVEMENT

The repaired implementation addresses the following possible problems in the original program:

- Boundary errors: prevent words from extending beyond the grid.
- Letter mismatch: prevent incompatible letters from occupying the same cell.
- Incorrect rollback: restore the grid to its previous state after an unsuccessful attempt.
- Invalid placements: check the selected position and orientation before placing a word.
- Incorrect termination: stop when a solution is found or all valid choices are exhausted.
- Input validation: handle empty word lists and words longer than the grid dimensions. The exact repairs should be documented based on the actual errors found in the original source code.

TABLE I
TEST CASES

1	Small list of compatible words	Valid crossword generated
2	Words with matching intersections	Shared letters match correctly
3	Word longer than the grid	Word rejected or no solution reported
4	Empty word list	Empty grid or appropriate message
5	Impossible word arrangement	Failure reported without crashing
6	Multiple possible placements	Valid arrangement found if one exists

VII. COMPLEXITY ANALYSIS

Let n be the number of words, P the maximum number of candidate placements considered for each word, and L the maximum word length. In a simple backtracking implementation, the worst-case search may examine up to Pn combinations of placements. If checking one placement requires O(L) time, a simplified upper bound on the running time is

$$T(n) = O(LPn) \tag{1}$$

The actual complexity depends on the grid size, placement rules, and pruning techniques; backtracking can have exponential worst-case time complexity. The grid requires O(RC) space for R rows and C columns, and the recursion adds up to O(n) depth.

VIII. ADVANTAGES AND LIMITATIONS

- Advantages
  - Automates crossword puzzle generation.
  - Demonstrates recursion and backtracking.
  - Checks constraints systematically and explores alternative arrangements.
  - Helps identify and correct invalid word placements.
  - Can be extended for larger and more complex puzzles
- Limitations
  - Large word lists may require considerable computation.
  - Some word combinations may not fit the selected grid.
  - Puzzle quality depends on the placement rules.
  - The basic implementation may not generate clues or guarantee a unique solution.
  - Performance depends on how effectively invalid placements are rejected early.

IX. Consclusion and Application

The generator applies to educational puzzle applications, word games and recreational software, algorithm and data-structure demonstrations, constraint-satisfaction research, and programming practice involving recursion and search.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein *et al.*, *Introduction to algorithms*. MIT press Cambridge, 1990, vol. 2.

[2] N. Kovač, K. Ratković, M. Simeunović, L.-C. W. Done, S. Kordić, and H. Farahani, "Introduction to python programming," in *Data Science in Psychology: Using Python in Psychological Research*. Springer, 2026, pp. 3–31.